

IF25-22017 PENGEMBANGAN APLIKASI MOBILE

Modul Belajar 6

Networking REST API dengan Ktor

Mengambil laporan dari server, menampilkan memuat, dan memberi pesan yang jelas saat internet kampus mati

Muhammad Habib Algifari, S.Kom., M.T.I.

Pendamping Pertemuan 6. Tahun 2026

Komponen	Keterangan
Pertemuan	6
Capaian	CPMK0501, menerapkan konsep pemrograman dalam bentuk aplikasi yang mengambil data dari REST API
Waktu belajar	sekitar 5 jam belajar mandiri, ditambah 2,5 jam di kelas

MODUL 6

Peta belajar

Ikuti urutan ini dari atas ke bawah. Setiap bagian memakai apa yang sudah dibahas di bagian sebelumnya.

Urutan	Bagian
1	Tentang modul ini
2	Masalah pembuka: laporan yang tidak pernah sampai ke petugas
3	Langkah 1: Satu permintaan, satu jawaban
4	Langkah 2: REST, alamat untuk benda dan method untuk aksi
5	Langkah 3: Kode status, siapa yang salah
6	Langkah 4: JSON dan data class yang cocok
7	Langkah 5: Dependency dan engine per platform
8	Langkah 6: Satu HttpClient untuk seluruh aplikasi
9	Langkah 7: GET dan POST dengan Ktor
10	Langkah 8: Dari exception ke pesan yang dimengerti
11	Langkah 9: Repository, sumber yang sama dengan isi dari server
12	Langkah 10: Tiga keadaan layar
13	Latihan mandiri
14	Bawa ke proyekmu
15	Rangkuman
16	Cek pemahaman
17	Bacaan lanjut dan persiapan minggu depan

PEMBUKA

Tentang modul ini

Modul ini menemani Pertemuan 6. Sampai Modul 5, LaporanKampus sudah punya lima layar yang tersambung. Tetapi semua laporannya masih buatan sendiri di memori. Di modul ini kita mengambil laporan dari server sungguhan dengan pustaka **Ktor**, lalu menampilkan tiga keadaan yang selalu dibutuhkan aplikasi berjaringan: memuat, berhasil, dan gagal.

Modul ini terdiri dari tiga bagian. Langkah 1 sampai 4 membahas bahasa yang dipakai aplikasi dan server: HTTP, REST, dan JSON. Langkah 5 sampai 8 membahas Ktor client untuk memanggil server.

Langkah 9 dan 10 membahas repository dan keadaan layar, sehingga sumber data bisa diganti tanpa mengubah layar.

Setelah menyelesaikan modul ini, kalian bisa

No	Kemampuan	Dibahas di
1	Menjelaskan endpoint, method, kode status, dan JSON yang dikirim server	Langkah 1 sampai 4
2	Mengambil dan mengirim data dengan Ktor dan <code>kotlinx.serialization</code>	Langkah 5 sampai 8
3	Menampilkan keadaan memuat, berhasil, dan gagal dengan tombol coba lagi lewat repository	Langkah 9 dan 10

Capaian ini mendukung **CPMK0501**, dalam bentuk aplikasi yang mengambil data dari REST API. Kemampuan ketiga juga yang diamati di observasi praktik minggu ini.

Yang perlu disiapkan

Proyek LaporanKampus dari Modul 5, koneksi internet, dan browser. Plugin `kotlinx-serialization` sudah terpasang sejak Modul 5 untuk rute bertipe, jadi tidak perlu dipasang ulang.

PENTING Mengingat modul sebelumnya

Kita memakai `suspend function`, `try` dan `catch`, serta `CancellationException` dari Modul 2, interface `Sumber<T>` dari Modul 2, `ViewModel` dan `StateFlow` dari Modul 4, serta sealed class untuk keadaan layar dari Modul 5. Kalau ada yang terasa asing, buka lagi bagian itu sebelum lanjut.

PEMBUKA

Masalah pembuka: laporan yang tidak pernah sampai ke petugas

Lima layar kita sudah tersambung. Tetapi semua laporan masih dibuat sendiri di memori HP. Petugas di gedung lain tidak pernah melihat laporan kalian, dan begitu aplikasi ditutup, laporan baru ikut hilang.

Kalian pasti pernah membuka aplikasi yang hanya menampilkan layar putih saat sinyal lemah, tanpa penjelasan apa pun. Pengguna jadi bingung: apakah aplikasinya rusak, atau masih menunggu?

TEBAK DULU **Coba pikirkan**

Saat aplikasi mengambil data dari server, apa saja yang seharusnya bisa tampil di layar?

JAWABAN

Tiga hal. Pertama, tanda **sedang memuat** selama menunggu jawaban. Kedua, **daftar laporan** kalau jawaban berhasil diterima. Ketiga, **pesan error yang jelas dengan tombol coba lagi** kalau gagal, misalnya karena internet kampus mati. Tiga keadaan inilah yang kita bangun di modul ini.

LANGKAH 1

Satu permintaan, satu jawaban

Aplikasi dan server berbicara dengan **HTTP**. Polanya selalu sama: aplikasi mengirim satu permintaan, server membalas dengan satu jawaban.

Permintaan dari aplikasi

```
GET /laporan HTTP/1.1
Host: alamat-server-kelompok
Accept: application/json
```

Jawaban dari server

```
HTTP/1.1 200 OK
Content-Type: application/json

[ { "id": 7, "judul": "AC mati" } ]
```

Bagian	Isinya	Di contoh ini
Method	Apa yang ingin dilakukan	GET, mengambil data
Endpoint	Alamat benda yang diminta	/laporan
Kode status	Hasilnya secara singkat	200 OK, berhasil
Body	Isi data, biasanya JSON	Daftar laporan

ANALOGI Loker layanan akademik

Kalian datang ke loket akademik membawa formulir berisi apa yang diminta, misalnya transkrip nilai. Petugas membalas dengan cap diterima atau ditolak, ditambah berkas yang diminta. Formulir itu adalah permintaan, cap itu adalah kode status, dan berkasnya adalah body.

PENTING Lihat sendiri

Buka situs apa saja di browser, tekan F12 untuk membuka DevTools, lalu pilih tab Network dan muat ulang halaman. Klik salah satu baris. Kalian akan melihat method, alamat, kode status, dan isi jawaban, persis bagian yang sama dengan tabel di atas.

LANGKAH 2

REST, alamat untuk benda dan method untuk aksi

REST adalah kebiasaan menyusun alamat server supaya mudah ditebak. Alamat menunjuk benda, dan method menunjuk aksi yang dilakukan pada benda itu.

Method	Endpoint	Artinya di LaporKampus
GET	/laporan	Ambil semua laporan
GET	/laporan/7	Ambil satu laporan dengan id 7
POST	/laporan	Kirim laporan baru, server memberi id
PUT	/laporan/7	Ganti isi laporan 7
DELETE	/laporan/7	Hapus laporan 7

Perhatikan bahwa endpoint memakai kata benda, bukan kata kerja. Tidak ada /ambilLaporan atau /hapusLaporan, karena aksinya sudah tertulis di method. Id di endpoint sama dengan id yang kita kirim lewat rute di Modul 5.

CONTOH Contoh sehari-hari

Bayangkan rak buku perpustakaan dengan nomor rak. Nomor rak adalah endpoint, dan kalian bisa melakukan beberapa aksi pada rak yang sama: melihat isi, menaruh buku baru, mengganti buku, atau mengeluarkan buku. Nomornya tidak berubah, yang berubah hanya aksinya.

LANGKAH 3

Kode status, siapa yang salah

Kode status adalah angka tiga digit yang meringkas hasil permintaan. Angka depannya paling penting.

Kelompok	Contoh	Artinya	Yang ditampilkan ke pengguna
2xx berhasil	200, 201	Permintaan diterima. 201 berarti data baru dibuat	Data
4xx salah di klien	400, 401, 404	Permintaan keliru, belum masuk, atau data tidak ditemukan	Pesan yang bisa diperbaiki pengguna
5xx salah di server	500, 503	Server error atau sedang tidak tersedia	Pesan untuk mencoba lagi nanti

Cara mengingatnya: angka depan **4** berarti kita yang perlu memperbaiki permintaan, angka depan **5** berarti server yang bermasalah. 404 termasuk kesalahan sisi klien, karena kita meminta sesuatu yang memang tidak ada.

ANALOGI Mengirim surat

Kode 4xx seperti surat yang kembali karena alamatnya salah tulis: yang perlu diperbaiki adalah pengirimnya. Kode 5xx seperti kantor pos yang sedang tutup: alamat kalian sudah benar, tinggal tunggu dan kirim lagi nanti.

LANGKAH 4

JSON dan data class yang cocok

Server mengirim data dalam format **JSON**: pasangan kunci dan nilai di dalam kurung kurawal.

Jawaban server

```
{
  "id": 7,
  "judul": "AC mati",
  "lokasi": "GKU 1 R201",
  "status": "Diproses",
  "petugas": "Pak Joko",
  "dibuatPada": "2026-10-05"
}
```

Untuk membaca JSON ini, kita butuh data class dengan bentuk yang cocok:

commonMain/data/LaporanDto.kt

```
import kotlinx.serialization.Serializable

@Serializable
data class LaporanDto(
    val id: Int,
    val judul: String,
    val lokasi: String,
    val status: String,
    val petugas: String? = null,
    val catatan: String? = null
)

@Serializable
data class LaporanBaruDto(val judul: String, val lokasi: String)
```

Ada tiga aturan penting. Pertama, `@Serializable` meminta plugin serialization membuatkan kode pembaca dan penulis JSON untuk kelas ini. Kedua, nama properti harus sama persis dengan kunci JSON. Ketiga, properti dengan nilai bawaan `= null` boleh tidak dikirim server.

DTO singkatan dari data transfer object, yaitu bentuk data persis seperti yang dikirim server. Kita sengaja memisahkannya dari data class Laporan milik aplikasi, karena status di server berupa teks biasa, sedangkan aplikasi kita memakai sealed class StatusLaporan. Penerjemahannya kita tulis di Langkah 9. LaporanBaruDto hanya berisi judul dan lokasi, karena id dibuat oleh server.

TEBAK DULU

Server mengirim kunci `dibuatPada`, padahal LaporanDto tidak punya properti itu. Apa yang terjadi saat JSON ini dibaca dengan pengaturan bawaan?

JAWABAN

Gagal. `kotlinx.serialization` melempar **`SerializationException`** karena ada kunci yang tidak dikenal. Pengaturan bawaan sengaja ketat supaya salah ketik nama properti ketahuan. Sebaliknya, kunci `catatan` yang tidak dikirim tidak membuat error, karena punya nilai bawaan null. Kalau yang hilang kunci `judul`, error juga, karena judul tidak punya nilai bawaan.

Server nyata hampir selalu mengirim lebih banyak kunci daripada yang kita butuhkan, dan bisa menambah kunci baru kapan saja. Karena itu hampir semua aplikasi memasang `ignoreUnknownKeys = true`, yang akan kita lihat di Langkah 6.

ANALOGI Formulir pendaftaran

DTO seperti formulir yang hanya punya kolom tertentu. Pengaturan ketat seperti petugas yang menolak formulir kalau ada tulisan tambahan di luar kolom. `ignoreUnknownKeys` seperti petugas yang mengabaikan coretan di pinggir formulir dan hanya membaca kolom yang ia butuhkan.

LANGKAH 5

Dependency dan engine per platform

Ktor adalah pustaka dari JetBrains untuk memanggil server. Kode pemanggilnya ditulis sekali di `commonMain`, tetapi koneksi sesungguhnya dibuka oleh **engine** yang berbeda di tiap platform.

`composeApp/build.gradle.kts`

```
kotlin {
    sourceSets {
        commonMain.dependencies {
            implementation(libs.ktor.client.core)
            implementation(libs.ktor.client.content.negotiation)
            implementation(libs.ktor.serialization.kotlinx.json)
        }
        androidMain.dependencies {
            implementation(libs.ktor.client.okhttp)
        }
        iosMain.dependencies {
            implementation(libs.ktor.client.darwin)
        }
        val desktopMain by getting
        desktopMain.dependencies {
            implementation(libs.ktor.client.cio)
        }
    }
}
```

Platform	Engine
Android	OkHttp, atau engine Android bawaan Ktor
iOS	Darwin
Desktop JVM	CIO, atau OkHttp

Kode di `commonMain` sama untuk semua platform. Engine adalah mesin yang benar-benar membuka koneksi. Pada template KMP yang lebih baru, source set desktop bernama `jvmMain`, jadi cukup tulis `jvmMain.dependencies { ... }` tanpa baris `val desktopMain by getting`. Yang penting, setiap source set punya satu engine. Kalau tidak, muncul error `No client engine found`.

ANALOGI Kurir paket

Kalian menulis alamat dan isi paket dengan cara yang sama, di mana pun kalian tinggal. Yang mengantar bisa berbeda: di satu kota memakai kurir A, di kota lain kurir B. Kode Ktor di `commonMain` adalah paketnya, engine adalah kurirnya.

PENTING Izin INTERNET di Android

Aplikasi Android tidak boleh memakai internet tanpa izin. Pastikan baris `<uses-permission android:name="android.permission.INTERNET" />` ada di `androidMain/AndroidManifest.xml`, di luar tag `application`. Proyek hands-on P6 memakai Ktor seri 3.0. Samakan versi Ktor di `libs.versions.toml` untuk semua anggota kelompok.

LANGKAH 6

Satu HttpClient untuk seluruh aplikasi

`commonMain/data/Jaringan.kt`

```
import io.ktor.client.HttpClient
import io.ktor.client.plugins.HttpTimeout
import io.ktor.client.plugins.contentnegotiation.ContentNegotiation
import io.ktor.serialization.kotlinx.json.json
import kotlinx.serialization.json.Json

fun buatClient() = HttpClient {
    install(ContentNegotiation) {
        json(Json { ignoreUnknownKeys = true })
    }
    install(HttpTimeout) {
        requestTimeoutMillis = 10_000
    }
    expectSuccess = true
}
```

Bagian	Gunanya
<code>ContentNegotiation</code> dengan <code>json(...)</code>	Mengubah JSON menjadi objek dan sebaliknya
<code>ignoreUnknownKeys = true</code>	Kunci JSON yang tidak dikenal diabaikan, tidak membuat error
<code>HttpTimeout</code>	Menyerah setelah 10 detik tanpa jawaban
<code>expectSuccess = true</code>	Kode status selain 2xx dianggap error

`HttpClient { }` tanpa menyebut nama engine akan memakai engine yang ada di dependency platform itu. Sepuluh detik adalah angka wajar untuk internet kampus yang kadang lambat. Kalian boleh memilih angka lain, asal ada.

PENTING Buat satu kali saja

Setiap `HttpClient` punya koneksi dan thread sendiri. Membuat client baru di setiap pemanggilan itu boros. Buat satu kali di tingkat aplikasi, lalu dipakai bersama oleh semua fungsi yang memanggil server.

LANGKAH 7

GET dan POST dengan Ktor

```
commonMain/data/LaporanApi.kt
```

```

import io.ktor.client.HttpClient
import io.ktor.client.call.body
import io.ktor.client.request.get
import io.ktor.client.request.post
import io.ktor.client.request.setBody
import io.ktor.http.ContentType
import io.ktor.http.contentType

// Isi dengan alamat API kelompok, tanpa garis miring di akhir.
const val BASE_URL = "https:// ..."

class LaporanApi(private val client: HttpClient) {
    suspend fun semua(): List<LaporanDto> =
        client.get("$BASE_URL/laporan").body()

    suspend fun ambil(id: Int): LaporanDto =
        client.get("$BASE_URL/laporan/$id").body()

    suspend fun kirim(baru: LaporanBaruDto): LaporanDto =
        client.post("$BASE_URL/laporan") {
            contentType(ContentType.Application.Json)
            setBody(baru)
        }.body()
}

```

`body()` membaca JSON menjadi tipe yang diminta fungsi. Tipe kembalian fungsi itulah yang menentukan hasilnya: `List<LaporanDto>` untuk semua, `LaporanDto` untuk satu laporan. Semua fungsi bertanda `suspend`, jadi harus dipanggil dari coroutine, sama seperti di Modul 2.

Alamat server disimpan di satu konstanta `BASE_URL`, bukan ditulis berulang di setiap fungsi. Kalau alamatnya pindah, cukup satu baris yang diubah. Kalau kelompok belum punya server, pakai API publik yang sesuai tema, atau mock API buatan sendiri.

TEBAK DULU

Seandainya `expectSuccess` tidak dipasang, lalu server menjawab kode 500 dengan halaman error HTML. Apakah `client.get` melempar exception? Lalu apa yang terjadi saat `body()` dipanggil?

JAWABAN

`client.get` tidak melempar apa pun, karena tanpa `expectSuccess` Ktor menganggap 500 tetap sebuah jawaban. Yang gagal justru `body()`, karena halaman error HTML tidak bisa dibaca sebagai daftar `LaporanDto`. Pesan errornya membingungkan, seolah-olah masalahnya ada di cara membaca data. Dengan `expectSuccess`, Ktor langsung melempar **`ServerResponseException`** yang jelas penyebabnya.

LANGKAH 8

Dari exception ke pesan yang dimengerti

Pengguna tidak perlu membaca nama exception. Mereka perlu tahu apa yang bisa dilakukan: menunggu, memeriksa sinyal, atau memperbaiki isian. Kita terjemahkan setiap jenis kegagalan menjadi kalimat yang jelas.

commonMain/data/Pesan.kt

```
import io.ktor.client.plugins.ClientRequestException
import io.ktor.client.plugins.HttpRequestTimeoutException
import io.ktor.client.plugins.ServerResponseException

fun pesanUntuk(e: Throwable): String = when (e) {
    is HttpRequestTimeoutException →
        "Server terlalu lama menjawab. Coba lagi."
    is ClientRequestException →
        if (e.response.status.value == 404) "Laporan tidak ditemukan."
        else "Permintaan ditolak server."
    is ServerResponseException →
        "Server sedang bermasalah. Coba lagi nanti."
    else → "Tidak bisa terhubung. Periksa internet."
}
```

Kejadian	Exception
Lebih dari 10 detik tanpa jawaban	HttpRequestTimeoutException
Kode 4xx	ClientRequestException
Kode 5xx	ServerResponseException
Tidak ada internet	Exception dari engine, berbeda per platform

Dua exception untuk kode status hanya muncul karena `expectSuccess = true`. Saat internet mati, jenis exception-nya berbeda di tiap engine, jadi cukup ditangkap di cabang else.

CONTOH Contoh sehari-hari

Bandingkan dua pesan ini: `java.net.UnknownHostException`, dan "Tidak bisa terhubung, periksa internet". Pesan pertama hanya berguna bagi programmer. Pesan kedua langsung memberi tahu pengguna apa yang harus dilakukan, misalnya menyalakan data seluler atau pindah ke tempat dengan Wi-Fi.

LANGKAH 9

Repository, sumber yang sama dengan isi dari server

Interface Sumber yang diperluas

Interface `Sumber<T>` dari Modul 2 kita tambah dua fungsi: `semua` untuk layar daftar dan `kirim` untuk form.

commonMain/data/Sumber.kt

```
interface Sumber<T> {
    suspend fun semua(): List<T>    // baru
    suspend fun ambil(id: Int): T
    fun pantau(id: Int): Flow<T>
    suspend fun kirim(baru: T): T    // baru
}
```

Sumber dari server

commonMain/data/SumberLaporanJaringan.kt

```
class SumberLaporanJaringan(private val api: LaporanApi) : Sumber<Laporan> {
    override suspend fun semua() = api.semua().map { it.keLaporan() }
    override suspend fun ambil(id: Int) = api.ambil(id).keLaporan()
    override fun pantau(id: Int) = flow { emit(ambil(id)) }
    override suspend fun kirim(baru: Laporan) =
        api.kirim(LaporanBaruDto(baru.judul, baru.lokasi)).keLaporan()
}
```

LaporanApi hanya berurusan dengan HTTP. SumberLaporanJaringan menerjemahkan hasilnya menjadi Laporan milik aplikasi. Penerjemahnya adalah extension function berikut:

commonMain/data/LaporanDto.kt

```
fun LaporanDto.keLaporan() = Laporan(
    id = id,
    judul = judul,
    lokasi = lokasi,
    fotoUrl = null,
    status = when (status) {
        "Diproses" → StatusLaporan.Diproses(petugas.orEmpty())
        "Selesai" → StatusLaporan.Selesai(catatan.orEmpty())
        else → StatusLaporan.Baru
    }
)
```

Teks status dari server diubah menjadi sealed class StatusLaporan dari Modul 2. `orEmpty()` mengubah null menjadi teks kosong. Cabang else menangani teks status yang tidak kita kenal,

misalnya kalau server menambah status baru. Aplikasi tidak crash, laporan itu dianggap Baru. fotoUrl sementara null, karena foto baru kita tangani di Modul 8.

Sumber palsu tetap berguna

Sumber palsu dari Modul 2 juga harus mengimplementasikan dua fungsi baru. Sekalian kita gabungkan dengan PenyimpananLaporan dari Modul 4, sehingga sumber palsu ini bisa dipakai saat server belum siap atau untuk pengujian di Modul 10.

commonMain/data/SumberLaporanPalsu.kt

```
class SumberLaporanPalsu : Sumber<Laporan> {
    private val daftar = MutableStateFlow(contoh)

    override suspend fun semua(): List<Laporan> {
        delay(500)
        return daftar.value
    }
    override suspend fun ambil(id: Int) = daftar.value.first { it.id == id }
    override fun pantau(id: Int) = daftar.map { semua → semua.first { it.id == id } }
    override suspend fun kirim(baruk: Laporan): Laporan {
        val tersimpan = baru.copy(id = daftar.value.size + 1)
        daftar.update { it + tersimpan }
        return tersimpan
    }
}
```

Inilah yang disebut **repository pattern**. Layar dan ViewModel hanya kenal Sumber. Mereka tidak tahu apakah datanya dari memori atau dari server.

ANALOGI Colokan listrik, lagi

Di Modul 2 kita memakai perumpamaan colokan listrik untuk interface. Sekarang terbukti manfaatnya. SumberLaporanPalsu dan SumberLaporanJaringan adalah dua alat berbeda yang sama-sama cocok dengan stopkontak Sumber. ViewModel adalah stopkontaknya, dan ia tidak peduli alat mana yang dicolokkan.

LANGKAH 10

Tiga keadaan layar

Keadaan sebagai sealed interface

commonMain/ui/DaftarUiState.kt

```
sealed interface DaftarUiState {
    data object Memuat : DaftarUiState
    data class Berhasil(val laporan: List<Laporan>) : DaftarUiState
    data class Gagal(val pesan: String) : DaftarUiState
}
```

Bentuknya mirip DetailUiState di Modul 5, hanya memakai `sealed interface`. Bedanya dengan sealed class kecil saja: interface tidak punya constructor, jadi turunannya cukup menulis : `DaftarUiState` tanpa kurung.

Kenapa tidak memakai tiga Boolean seperti `memuat`, `gagal`, dan `berhasil`? Bandingkan:

	Tiga Boolean	Sealed interface
Memuat sekaligus gagal	Bisa terjadi	Tidak mungkin
Data saat gagal	Daftar lama bisa tertinggal	Tidak ada daftar
when lupa satu kasus	Tidak ketahuan	Error saat compile

Tiga Boolean punya delapan kombinasi, padahal yang masuk akal hanya tiga. Sealed interface membuat keadaan yang mustahil tidak bisa ditulis sama sekali.

ANALOGI Lampu lalu lintas

Lampu lalu lintas hanya punya tiga keadaan, dan hanya satu yang menyala pada satu waktu. Bayangkan kalau lampu merah dan hijau bisa menyala bersamaan karena masing-masing punya saklar sendiri. Itulah tiga Boolean. Sealed interface seperti lampu yang dirancang sehingga hanya satu warna yang bisa menyala.

ViewModel yang memuat dari server

```
commonMain/ui/DaftarViewModel.kt
```

```

class DaftarViewModel(private val sumber: Sumber<Laporan>) : ViewModel() {
    private val _uiState = MutableStateFlow<DaftarUiState>(DaftarUiState.Memuat)
    val uiState = _uiState.asStateFlow()

    init { muat() }

    fun muat() {
        viewModelScope.launch {
            _uiState.value = DaftarUiState.Memuat
            _uiState.value = try {
                DaftarUiState.Berhasil(sumber.semua())
            } catch (e: CancellationException) {
                throw e
            } catch (e: Exception) {
                DaftarUiState.Gagal(pesanUntuk(e))
            }
        }
    }
}

```

`muat()` dipanggil sekali di blok `init`, yaitu saat ViewModel dibuat, lalu dipanggil lagi oleh tombol Coba lagi. Setiap kali dipanggil, keadaan kembali ke Memuat dulu, sehingga pengguna tahu tombolnya bekerja. `try` menghasilkan nilai, jadi hasilnya langsung menjadi `uiState`. Pola `CancellationException` yang dilempar ulang sama dengan Modul 2.

Layar memilih tampilan dengan when

commonMain/ui/DaftarScreen.kt

```

@Composable
fun DaftarScreen(vm: DaftarViewModel, bukaDetail: (Int) → Unit, bukaForm: () → Unit) {
    {
        val state by vm.uiState.collectAsState()
        Box(modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
            when (val s = state) {
                DaftarUiState.Memuat → CircularProgressIndicator()
                is DaftarUiState.Berhasil → LayarDaftar(s.laporan, bukaDetail, bukaForm)
                is DaftarUiState.Gagal → PesanGagal(s.pesan, vm::muat)
            }
        }
    }
}

@Composable
fun PesanGagal(pesan: String, onCobaLagi: () → Unit) {
    Column(horizontalAlignment = Alignment.CenterHorizontally) {
        Text(pesan)
        Button(onClick = onCobaLagi) { Text("Coba lagi") }
    }
}

```

`when (val s = state)` membuat smart cast bekerja, sehingga `s.laporan` dan `s.pesan` bisa dibaca langsung. Satu perubahan kecil: `LayarDaftar` dari Modul 5 sekarang menerima daftar sebagai parameter pertama, bukan membaca `PenyimpananLaporan` sendiri. Kalau besok kita menambah keadaan keempat, misalnya Kosong, compiler langsung menunjuk `when` ini.

Coba alurnya tanpa server

Jalankan simulasi ini di Kotlin Playground. Server dibuat pura-pura, sehingga tidak butuh internet.

Main.kt

```
import kotlinx.coroutines.*

sealed interface DaftarUiState {
    data object Memuat : DaftarUiState
    data class Berhasil(val laporan: List<String>) : DaftarUiState
    data class Gagal(val pesan: String) : DaftarUiState
}

var ke = 0
suspend fun serverPalsu(): List<String> {
    ke++
    delay(500)
    if (ke == 1) throw Exception("internet mati")
    return listOf("AC mati", "Proyektor rusak")
}

fun main() = runBlocking {
    repeat(2) {
        println(DaftarUiState.Memuat)
        val hasil = try {
            DaftarUiState.Berhasil(serverPalsu())
        } catch (e: Exception) {
            DaftarUiState.Gagal("Tidak bisa terhubung. Periksa internet.")
        }
        println(hasil)
    }
}
```

TEBAK DULU

Tulis empat baris keluarannya. Lalu, apa yang berubah kalau `ke == 1` diganti menjadi `ke ≤ 2`?

JAWABAN

Keluaran

```
Memuat
Gagal(pesan=Tidak bisa terhubung. Periksa internet.)
Memuat
Berhasil(laporan=[AC mati, Proyektor rusak])
```

Percobaan pertama gagal, lalu percobaan kedua yang mewakili tombol Coba lagi berhasil. Kalau diganti `ke ≤ 2`, kedua percobaan gagal, sehingga Gagal tercetak dua kali. Perhatikan bahwa setiap permintaan selalu diawali Memuat dan berakhir di Berhasil atau Gagal, tidak pernah menggantung.

Form akhirnya mengirim ke server

Fungsi `kirim` di `FormLaporanViewModel` dari Modul 4 kini memanggil `sumber.kirim`, menggantikan `delay(800)`. `ViewModel` menerima sumber lewat constructor, sama seperti `DaftarViewModel`.

commonMain/ui/FormLaporanViewModel.kt

```
fun FormLaporanUiState.keLaporan() = Laporan(0, judul, lokasi, null)

fun kirim() {
    val s = _uiState.value
    if (!s.bisaKirim) return
    viewModelScope.launch {
        _uiState.value = s.copy(sedangMengirim = true)
        _uiState.value = try {
            sumber.kirim(s.keLaporan())
            FormLaporanUiState(pesan = "Laporan terkirim")
        } catch (e: CancellationException) {
            throw e
        } catch (e: Exception) {
            s.copy(pesan = pesanUntuk(e))
        }
    }
}
```

id diisi 0, karena id aslinya dibuat server. Kalau gagal, isi form tidak dikosongkan. Pengguna cukup menekan Kirim lagi tanpa mengetik ulang.

PENTING Pesan gagal jangan memicu kembali ke daftar

Di Modul 5, `FormLaporanScreen` kembali ke daftar setiap kali `state.pesan` tidak null. Sekarang pesan juga dipakai untuk kegagalan. Ubah syaratnya menjadi `if (state.pesan == "Laporan terkirim") selesai()`, atau tambahkan properti `terkirim: Boolean` di `UiState` supaya lebih rapi.

Mengganti sumber hanya dengan satu baris

Buat sumber satu kali di tingkat aplikasi, lalu berikan ke setiap `ViewModel`:

commonMain/App.kt

```
@Composable
fun LaporanKampusApp() {
    val sumber: Sumber<Laporan> = remember {
        SumberLaporanJaringan(LaporanApi(buatClient())) }
    // sebelumnya: remember { SumberLaporanPalsu() }
    val nav = rememberNavController()
    NavHost(nav, startDestination = RuteBeranda) {
        composable<RuteBeranda> {
            val vm = viewModel { DaftarViewModel(sumber) }
            DaftarScreen(vm,
                bukaDetail = { id → nav.navigate(RuteDetail(id)) },
                bukaForm = { nav.navigate(RuteForm) })
        }
        // RuteDetail dan RuteForm memakai sumber yang sama
    }
}
```

Hanya baris pembuatan sumber yang berubah. Layar daftar, detail, dan form tidak disentuh sama sekali. Inilah buah dari MVVM di Modul 4 dan interface di Modul 2. Di Modul 10, pemberian sumber ke setiap ViewModel ini akan dirapikan dengan Koin.

PRAKTIK

Latihan mandiri

Ada tiga latihan di folder P6 Networking REST API Hands-on. Semuanya memanggil `jsonplaceholder.typicode.com/users`, yaitu API latihan gratis tanpa kunci, jadi butuh internet. Latihan 2 dan 3 yang diamati.

Latihan	Berkas	Yang dilatih
1	Handson1KtorClient.kt	HttpClient dengan ContentNegotiation, lalu <code>fetchUsers</code> yang membaca daftar <code>RemoteUser</code>
2	Handson2Repository.kt	UserRepository dan <code>load()</code> yang mengubah keadaan Loading, Success, atau Error
3	Handson3UserListUi.kt	Indikator memuat, LazyColumn, dan pesan error dengan tombol Coba Lagi

CONTOH Petunjuk latihan 1

`RemoteUser` hanya punya tiga properti, padahal API latihan mengirim jauh lebih banyak kunci. Tanpa `ignoreUnknownKeys = true`, kalian akan bertemu `SerializationException` dari Langkah 4. Cetak hasilnya dengan `println` dulu sebelum menampilkannya di layar.

CONTOH Petunjuk latihan 2

Polanya sama dengan `DaftarViewModel` di Langkah 10, hanya di sana kelas biasa bernama `UserListStateHolder`. Pastikan setiap kemungkinan berakhir di Success atau Error. Uji dengan mematikan Wi-Fi laptop sebentar: keadaan Error harus benar-benar terisi, bukan diam di Loading. Untuk tantangan, ganti sealed class di latihan ini menjadi sealed interface.

CONTOH Petunjuk latihan 3

Tiru `DaftarScreen` di Langkah 10: satu `when` dengan tiga cabang. Tombol Coba Lagi memanggil ulang fungsi `load`. Pusatkan indikator dan pesan dengan `Box` dan `Alignment.Center` supaya rapi.

Yang diamati

Level	Skor	Yang terlihat
Sangat baik	85 sampai 100	Data tampil lewat repository, keadaan memuat dan error tampil, tombol Coba Lagi berjalan

Level	Skor	Yang terlihat
Baik	70 sampai 84	Data tampil, satu keadaan belum ditangani
Cukup	55 sampai 69	Data tampil tanpa keadaan memuat dan error
Kurang	di bawah 55	Belum memanggil API

Observasi dikerjakan individu dengan bobot 4% dari nilai akhir, untuk CPMK0501. Buktinya: push latihan ke repository latihan pribadi, lalu kirim tautan commit di LMS sebelum kelas selesai. Kalau internet kampus lambat, jalankan target desktop karena lebih cepat daripada emulator.

Kalau error

Gejala atau pesan	Penyebab	Yang dicoba
Permission denied, missing INTERNET permission	Izin belum ada di AndroidManifest.xml	Tambahkan uses-permission INTERNET
No client engine found	Engine untuk platform itu belum ada	Tambahkan engine di setiap source set
SerializationException	Nama properti tidak cocok atau ada kunci tambahan	Cocokkan nama, pasang ignoreUnknownKeys
Layar tidak pernah keluar dari Memuat	Fungsi muat tidak dipanggil, atau exception tertelan	Panggil di init atau LaunchedEffect, dan pastikan setiap catch mengisi keadaan Gagal
NoTransformationFoundException	Server menjawab bukan JSON, sering kali halaman error	Pasang expectSuccess, periksa alamat di browser

PRAKTIK

Bawa ke proyekmu

Kebutuhan Bank Proyek menyebut REST API dengan Ktor, termasuk keadaan memuat dan error. Bagi kerja: satu anggota DTO dan Api, satu anggota Sumber, satu anggota ViewModel dan layar.

- Pilih servernya.** API publik yang sesuai tema, atau mock API buatan kelompok sendiri. Catat pilihan itu di README.
- DTO dan Api.** Data class `@Serializable` dan kelas Api dengan GET dan POST.
- Sumber jaringan.** Implementasi Sumber untuk entitas utama, layar tidak diubah.
- Tiga keadaan.** Memuat, berhasil, dan gagal dengan tombol coba lagi di layar daftar.

CONTOH Kalau tidak ada API yang cocok

Mock API sudah cukup. Bentuknya bisa berkas JSON yang disajikan layanan mock gratis, atau server kecil buatan sendiri. Yang penting endpointnya mengikuti kebiasaan REST di Langkah 2 dan jawabannya JSON. Di LaporKampus, contohnya adalah LaporanDto, LaporanApi, SumberLaporanJaringan, DaftarUiState, dan BASE_URL yang disimpan di satu konstanta.

Checkpoint minggu 6

No	Yang diselesaikan
1	REST API publik atau mock API buatan sendiri, dipanggil dengan Ktor
2	Implementasi Sumber untuk entitas utama, layar tetap memakai ViewModel
3	Tiga keadaan tampil, error punya pesan jelas dan tombol coba lagi

Push ke repository kelompok. README memuat alamat API yang dipakai dan tangkapan layar keadaan memuat dan error. Checkpoint tidak masuk nilai, tetapi menjadi bekal tes lisan di pertemuan 12 sampai 16. Data REST dan keadaan error termasuk fitur utama yang diperiksa di tes lisan 1.

PENUTUP

Rangkuman

Aplikasi dan server berbicara dengan **HTTP**: satu permintaan berisi method dan endpoint, satu jawaban berisi kode status dan body. **REST** memakai endpoint untuk benda dan method untuk aksi. Kode **4xx** berarti permintaan kita yang salah, **5xx** berarti server yang bermasalah.

Data dikirim sebagai **JSON** dan dibaca ke data class `@Serializable` yang disebut DTO. Nama properti harus sama dengan kunci JSON, dan `ignoreUnknownKeys` membuat kunci tambahan diabaikan.

Ktor memanggil server dari `commonMain` dengan engine yang berbeda di tiap platform. Satu `HttpClient` dibuat sekali, dengan `ContentNegotiation`, `timeout`, dan `expectSuccess = true`. Setiap exception diterjemahkan menjadi pesan yang dimengerti pengguna.

Repository pattern membuat `ViewModel` hanya kenal interface `Sumber`, sehingga sumber palsu bisa diganti sumber jaringan dengan satu baris. DTO diterjemahkan menjadi Laporan milik aplikasi.

Keadaan layar ditulis sebagai **sealed interface** `Memuat`, `Berhasil`, dan `Gagal`, sehingga keadaan mustahil tidak bisa terjadi. Setiap permintaan diawali `Memuat` dan selalu berakhir di `Berhasil` atau `Gagal`, dengan tombol coba lagi.

PENUTUP

Cek pemahaman

Jawab tanpa melihat modul dulu, lalu cocokkan dengan kunci di bawahnya.

1. Apa arti kode status 404 dan 500, dan mana yang kesalahan di sisi klien?
2. Kenapa endpoint REST untuk menghapus laporan 7 ditulis DELETE /laporan/7, bukan GET /hapusLaporan/7?
3. Kenapa data class butuh `@Serializable`, dan apa yang terjadi bila server mengirim kunci tambahan?
4. Kenapa kita memisahkan LaporanDto dari Laporan?
5. Kenapa HttpClient dibuat sekali, bukan di setiap pemanggilan?
6. Kenapa keadaan layar ditulis sebagai sealed interface, bukan tiga Boolean?
7. Sumber data diganti dari SumberLaporanPalsu menjadi SumberLaporanJaringan. Berapa layar yang harus diubah?

JAWABAN Kunci jawaban

1. 404 berarti data atau alamat tidak ditemukan dan termasuk kesalahan sisi klien. 500 berarti server mengalami error.
2. Karena REST memakai endpoint untuk benda dan method untuk aksi. Aksi hapus sudah dinyatakan oleh method DELETE.
3. Plugin serialization butuh anotasi itu untuk membuat kode pembaca dan penulis JSON. Kunci tambahan membuat SerializationException, kecuali ignoreUnknownKeys dipasang.
4. Supaya bentuk data server tidak bocor ke layar. Status di server berupa teks, sedangkan aplikasi memakai sealed class. Kalau server mengganti nama kunci, cukup fungsi keLaporan yang diubah.
5. Setiap client punya koneksi dan thread sendiri, jadi membuat banyak client itu boros.
6. Supaya keadaan mustahil seperti memuat sekaligus gagal tidak bisa terjadi, dan supaya when memeriksa semua kemungkinan.
7. Tidak ada. Hanya baris pembuatan sumber di App.kt yang berubah, karena layar dan ViewModel hanya kenal interface Sumber.

PENUTUP

Bacaan lanjut dan persiapan minggu depan

Sumber	Untuk apa
ktor.io, Ktor Client	Dependency, engine, dan plugin client
ktor.io, Content negotiation and serialization	Membaca dan mengirim JSON
ktor.io, Response validation	expectSuccess dan exception kode status
kotlinlang.org, Create a multiplatform app using Ktor and SQLDelight	Tutorial resmi yang juga dipakai di Modul 7
developer.mozilla.org, HTTP response status codes	Daftar lengkap kode status

CONTOH Masalah Pertemuan 7

Di gedung kuliah sinyal sering hilang. Begitu offline, daftar laporan kosong dan draf yang sedang ditulis ikut hilang. Coba sekarang: nyalakan mode pesawat di emulator lalu buka ulang LaporKampus. Yang tampil hanya keadaan Gagal, padahal semenit lalu daftarnya ada. Minggu depan kita menyimpan data itu di database lokal dengan SQLDelight.